

SLIMQ: Secure Lightweight Online/Offline Multi-Dimensional Range Query in Cloud

Tianyu Wei , Yandong Zheng , *Member, IEEE*, Hui Zhu , *Senior Member, IEEE*, Xue Yang , *Member, IEEE*, and Songnian Zhang , *Member, IEEE*

Abstract—The rapid growth of data-intensive applications has led to the ubiquity of multi-dimensional datasets and the increasing demand for range queries. Constrained by local storage and computational capabilities, outsourcing range query services to cloud servers has attracted considerable attention, while it raises serious privacy concerns when sensitive data are outsourced. Existing privacy-preserving range query schemes still have some limitations in single-dimensional query privacy preservation and computational efficiency. To address this issue, we propose a secure lightweight online/offline multi-dimensional range query scheme, named SLIMQ. First, we organize the dataset using an R-tree, encode a given multi-dimensional range query into a prefix set via prefix encoding, and reduce the processing of R-tree-based multi-dimensional range queries to a prefix matching problem. Then, we introduce an adaptive three-party secret sharing scheme. Based on this scheme, we design a private prefix matching algorithm Π_{PMatch} to determine whether a numerical value matches a prefix-encoded value. Furthermore, we construct a private point intersection test algorithm Π_{PInter} and a private range intersection test algorithm Π_{RInter} on the basis of Π_{PMatch} . All these algorithms adopt an offline–online mode, in which computation-intensive tasks are shifted to the offline phase and the online phase only involves lightweight operations. After that, we construct our SLIMQ scheme based on Π_{PInter} and Π_{RInter} . In addition, a simulation-based security analysis demonstrates that SLIMQ can protect data privacy and full single-dimensional query privacy. Extensive experimental evaluations show that SLIMQ significantly outperforms the state-of-the-art schemes PRQ and TRQED+ in terms of online query efficiency, while maintaining competitive offline precomputation overhead.

Index Terms—Multi-dimensional range query, R-tree, prefix encoding, adaptive secret sharing, single-dimensional query privacy.

I. INTRODUCTION

THE rapid proliferation of data-intensive applications, spanning healthcare systems [1], [2], financial analytics,

Received 15 April 2026; revised 14 June 2026; accepted 16 July 2026. Date of publication 20 July 2026; date of current version 1 September 2026. This work was supported in part by the National Cryptologic Science Fund of China under Grant 2025NCSF02015, in part by NSFC under Grant 62302374 and Grant U22B2030, in part by China Postdoctoral Science Foundation under Grant 2022M722498, and in part by the Shaanxi Provincial Preferential Funding Program for Returned Overseas Scholars under Grant 2023006. (Corresponding author: Yandong Zheng.)

Tianyu Wei, Yandong Zheng, Hui Zhu, and Songnian Zhang are with the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an 710071, China (e-mail: weitianyu@stu.xidian.edu.cn; zhengyandong@xidian.edu.cn; zhuhui@xidian.edu.cn; zhangsongnian@xidian.edu.cn).

Xue Yang is with the School of Information Science and Technology, Southwest Jiaotong University, Chengdu 611756, China (e-mail: xueyang@swjtu.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3715456

location-based services, and Internet of Things (IoT) ecosystems [3], [4], has led to the accumulation of large-scale multi-dimensional datasets. Effective querying and analysis of these datasets can uncover valuable insights and benefit the aforementioned applications, and range queries are among the most common query types. For example, by performing a range query on a healthcare dataset for individuals simultaneously satisfying constraints on age, blood pressure, cholesterol, and blood glucose levels, similar patients can be identified to assist in personalized treatment planning. However, processing such queries over large-scale datasets imposes prohibitive storage and computation burdens on the data owner side. To alleviate these burdens, data outsourcing to cloud computing platforms has become a practical solution [5]. While cloud outsourcing offers compelling advantages in storage and computation capabilities, it introduces serious privacy concerns due to the untrusted nature of cloud servers. To protect data privacy, the data owner usually encrypts the outsourced dataset using cryptographic techniques and designs secure schemes to achieve efficient query processing over encrypted data [6].

In the literature, considerable research efforts have been made to achieve efficient multi-dimensional range queries over encrypted data, but they exhibit fundamental limitations. Specifically, order-preserving encryption [7], [8], [9] can naturally support range queries, as it allows direct comparison of ciphertexts. Nevertheless, such schemes suffer from severe privacy leakage due to their inherent order-preserving property, which exposes statistical distributions of plaintext data. To mitigate such leakages, some bucketization-based schemes [10], [11] were proposed, in which plaintext data are assigned to buckets in a manner that the relative order of records within a single bucket remains confidential. However, such schemes impose most of the computational costs on resource-constrained query users. To address this issue, some studies employ public-key cryptography to construct privacy-preserving multi-dimensional range query schemes [12], [13], [14]. Since the security of these schemes relies on the computationally intensive bilinear pairing-based public-key encryption technique, they are not suitable for large-scale practical applications.

To further improve query efficiency, researchers have proposed a series of more lightweight solutions [15], [16], [17]. Specifically, the scheme in [15] employs symmetric-key range predicate encryption to protect data privacy and uses a segment tree structure to encode the numerical domain, thereby enabling secure and efficient single-dimensional range queries with

logarithmic complexity. In [16], asymmetric scalar-product-preserving encryption (ASPE) is combined with an R-tree index structure to support multi-dimensional range queries. The work in [17] constructs an encrypted interval tree index structure and integrates ASPE with noise [18] to protect data privacy, thereby enabling flexible sub-range queries in multi-user scenarios. Although these schemes significantly improve query performance by reducing expensive public-key operations and introducing efficient index structures, most of them reveal the range query results for each individual dimension, i.e., single-dimensional query information, leading to more information leakage compared to only disclosing multi-dimensional query results.

To protect single-dimensional query privacy, the TRQED scheme in [19] integrates an R-tree index to support multi-dimensional range queries and utilizes matrix encryption with fixed dimension permutation to achieve secure multi-dimensional range queries with partial single-dimensional query privacy. The TRQED+ scheme [20] further improves TRQED by leveraging a secure scalar product scheme [21] and an oblivious R-tree traversal algorithm. The scheme can not only protect data privacy and query privacy, but also additionally conceal the access patterns during the query process. The PRQ scheme in [22] leverages a data-encoding technique to transform data comparisons into equality tests and integrates matrix encryption to design full single-dimensional privacy-preserving point/range intersection test protocols for secure R-tree-based multi-dimensional range queries. However, the computational cost of this scheme strongly depends on the size of the numerical domain and requires a predefined domain parameter N to construct the encoding matrix, where N^2 represents the upper bound of the numerical domain. When the data domain becomes large, the matrix size grows significantly, leading to substantial storage and computational overhead. As a result, it is still challenging to design secure multi-dimensional range queries with high query efficiency and full single-dimensional query privacy.

To address this challenge, we propose a secure lightweight online/offline multi-dimensional range query scheme over encrypted data, named SLIMQ, that achieves full single-dimensional query privacy protection and high online computational efficiency. Our contributions are threefold as follows.

- First, to improve query efficiency, we index the dataset using an R-tree and encode the entries stored in both internal and leaf nodes into binary form. Then, we employ a prefix encoding technique in [23] to represent a given multi-dimensional range query as a prefix set. After that, we transform the processing of an R-tree-based multi-dimensional range query request into a prefix matching problem, thereby enabling efficient multi-dimensional range query processing.

- Second, we introduce an adaptive three-party secret sharing scheme based on the scheme in [24], which is adaptively designed for outsourced query-processing scenarios. Based on this scheme, we design a private prefix matching algorithm Π_{PMatch} to determine whether a numerical value matches a prefix-encoded value. Furthermore, we construct a private point intersection test algorithm Π_{PInter} and a private range intersection test algorithm Π_{RInter} on the basis of Π_{PMatch} , which serve as the two fundamental building blocks for securely searching

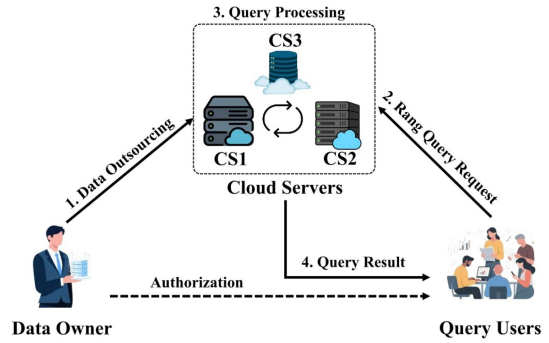


Fig. 1. Multi-dimensional range query model in the cloud.

a prefix-encoding-based R-tree. In addition, all the above algorithms provide full single-dimensional query privacy preservation and adopt an offline–online mode, in which computation-intensive tasks are shifted to the offline phase, while the online phase only involves lightweight operations.

- Third, by leveraging Π_{PInter} and Π_{RInter} , we construct our SLIMQ scheme. Meanwhile, a rigorous simulation-based security analysis demonstrates that SLIMQ can protect the privacy of outsourced datasets and query requests as well as full single-dimensional query privacy. Furthermore, extensive experimental evaluations are conducted on a real-world medical dataset. The results show that SLIMQ significantly outperforms the state-of-the-art schemes PRQ [22] and TRQED+ [20] in terms of online query efficiency, while maintaining competitive offline precomputation overhead.

Roadmap: The remainder of this paper is structured as follows. Section II formalizes the models and design goals. Section III reviews essential preliminaries including R-trees, prefix encoding, and three-party secret sharing. Section IV-D first introduces prefix-encoding-based multi-dimensional range queries, then presents an adaptive three-party secret sharing and, based on it, proposes three private algorithms: Π_{PMatch} , Π_{PInter} , and Π_{RInter} . Finally, the section presents our SLIMQ scheme built upon these algorithms. Section V provides rigorous security analysis. Section VI reports comprehensive experimental results. Section VII discusses related work, and Section VIII concludes the paper.

II. MODELS AND DESIGN GOALS

This section first presents the system model and the security model, and then specifies the design goals.

A. System Model

As shown in Fig. 1, our work focuses on range queries in the cloud, and the system contains the following types of entities.

- **Data Owner:** The data owner possesses a multi-dimensional dataset $\mathcal{X} = \{\mathbf{x}_j = (x_{j,1}, x_{j,2}, \dots, x_{j,d})\}_{j=1}^n$. Due to limited local computational resources, the data owner outsources the dataset to the cloud to enable efficient query processing and data management. To support secure and efficient query execution, the outsourced dataset is first indexed using a tree-based

structure to improve query performance, and then encrypted to ensure data privacy.

- *Three Cloud Servers:* The system includes three cloud servers, named CS_1 , CS_2 , and CS_3 . Each of them is equipped with sufficient storage and computational resources to store the outsourced dataset, receive range query requests, and execute the proposed scheme to process the queries. This three-server setting follows the common deployment model of three-party secret sharing and can be instantiated by independent cloud providers, institutional clouds, or cloud-edge nodes managed by different administrative domains.

- *Query Users:* The system supports multiple query users, each of whom must obtain authorization from the data owner before obtaining query credentials and accessing the system's functionalities. An authorized query user can launch a range query request $\mathbf{q} = ([q_{1,l}, q_{1,r}], \dots, [q_{d,l}, q_{d,r}])$, where each interval $[q_{i,l}, q_{i,r}]$ defines the query range in the i -th dimension for $i \in \{1, 2, \dots, d\}$. To preserve query privacy, the user encrypts the query vector using a predefined encryption scheme and submits the encrypted query to the cloud servers. After that, the cloud servers return a query response to the query user, and the user can locally reveal the desired query results.

B. Security Model

Since the dataset is held by the data owner, we assume that the data owner is trustworthy. The servers are assumed to be semi-honest. Query users are assumed to be honest and authorized by the data owner before obtaining query credentials. In practical deployments, misuse by authorized users can be mitigated through credential management, access-control policies, query quotas, authenticated logging, and auditing. These operational mechanisms complement the cryptographic privacy guarantees of SLIMQ, while resistance against malicious query users is outside the current semi-honest model. Besides, there are three non-collusion assumptions: i) no collusion among the three servers, which can be supported by selecting independent service providers or administrative domains with separated management, contractual obligations, and audit mechanisms; ii) no collusion between cloud servers and query users, which can be guaranteed by imposing severe penalties on any collusive behavior; and iii) no collusion between cloud servers and the data owner because their collusion will inevitably violate the dataset privacy [25], [26].

C. Design Goals

In this work, our goal is to design an efficient and privacy-preserving multi-dimensional range query scheme that supports offline precomputation and satisfies the following objectives:

- *Privacy Preservation:* In our scheme, the fundamental requirement is to protect data privacy from leakage. Specifically, the outsourced data, query requests, and the final query results must all remain confidential from cloud servers. Moreover, single-dimensional query privacy must also be preserved.
- *Efficiency:* To minimize the computational overhead of privacy protection, we need to design a strategy that supports offline precomputation, wherein complex and computationally

TABLE I
SUMMARY OF NOTATIONS

$\mathcal{X} = \{\mathbf{x}_j = (x_{j,1}, \dots, x_{j,d})\}_{j=1}^n$	Multi-dimensional dataset
$n = \mathcal{X} $	Number of data records in $\mathcal{X}$
N	Square root of the upper bound of the numerical domain, used in PRQ and LSED+
d	Number of dimensions
T	R-tree index built over $\mathcal{X}$
$T.root$	Root node of the R-tree T
$\mathbf{B} = (B_1, \dots, B_d)$	MBR of a tree node
$B_i = [b_{i,l}, b_{i,r}]$	The i -th dimensional interval of $\mathbf{B}$
$\mathbf{x} = (x_1, \dots, x_d)$	d -dimensional data point in a leaf node
$\mathbf{q}$	Multi-dimensional range query
$\mathcal{A}$	Set of data points satisfying the query $\mathbf{q}$
w	Bit-length of numeric representation
$\mathbb{Z}_2$	Binary ring for secret sharing
$\llbracket x \rrbracket$	Three-party secret sharing of bit x over $\mathbb{Z}_2$
$*$	Wildcard symbol in prefix encoding
$\mathcal{P}_R$	Minimal prefix cover of range R
Π_{PMatch}	Private prefix matching algorithm
Π_{PInter}	Private point intersection test algorithm
Π_{RInter}	Private range intersection test algorithm
Π_{aTPSS}	Adaptive three-party secret sharing scheme
$\llbracket \mathbf{q} \rrbracket^{\text{PInter}}$	Query token for private point intersection test
$\llbracket \mathbf{q} \rrbracket^{\text{RInter}}$	Query token for private range intersection test
$\mathcal{L}^{\text{pub}}$	Public leakage in the security analysis
S_i	Simulator for the view of cloud server CS_i

intensive operations are performed in the offline phase and can be precomputed in advance. The online phase then handles only lightweight computations, ensuring high efficiency for online query processing.

III. PRELIMINARIES

In this section, we provide an overview of several key concepts that form the foundation of our scheme, including R-trees, prefix encoding, and three-party secret sharing. For clarity, a summary of the notations is presented in Table I.

A. R-Tree

An R-tree is a height-balanced tree structure designed for the efficient indexing of multi-dimensional spatial data [27]. It uses Minimum Bounding Rectangles (MBRs) to organize data, where an MBR is the smallest axis-aligned rectangle that encloses all range records in the group. For example, if three two-dimensional range records $\{[1, 6], [3, 7], [2, 8]\}$ are grouped together, they can be represented by an MBR $B = [R_1, R_2]$, where $R_1 = [1, 3]$ and $R_2 = [6, 8]$. The R-tree is typically constructed in a bottom-up fashion: adjacent child nodes are grouped together, and their parent nodes are recursively formed by computing the MBRs that enclose their respective children, continuing until a single root node is created. This hierarchical construction

Algorithm 1: Range Query in R-Tree.

Input: R-tree root node T , range query $\mathbf{q} = ([q_{1,l}, q_{1,r}], \dots, [q_{d,l}, q_{d,r}])$

Output: Set of points $\mathcal{A}$ within the query range

Initialize the result set $\mathcal{A} \leftarrow \emptyset$;

Initialize an empty stack $S = \emptyset$;

$S.push(T.root)$

while S is not empty **do**

Pop a node $Node$ from S ;

if $Node$ is an internal node with $(\mathbf{B}, \text{ptr})$ **then**

if $\mathbf{B}$ intersects with $\mathbf{q}$ in all dimensions, i.e., $[q_{i,l}, q_{i,r}] \cap B_i \neq \emptyset$ for all $i \in \{1, 2, \dots, d\}$ **then**

Push all child nodes of $Node$ to S ;

else if $Node$ is a leaf node with $\mathbf{x} = (x_1, x_2, \dots, x_d)$ **then**

if $x_i \in [q_{i,l}, q_{i,r}]$ for all $i \in \{1, 2, \dots, d\}$ **then**

$\mathcal{A} \leftarrow \mathcal{A} \cup \{\mathbf{x}\}$;

return $\mathcal{A}$;

preserves spatial locality, which is critical for enabling effective query pruning and improving query performance.

In our proposed scheme, each internal node of the R-tree is represented as $(\mathbf{B}, \text{ptr})$, where $\mathbf{B} = (B_1, B_2, \dots, B_d)$ denotes the Minimum Bounding Rectangle (MBR) encompassing all data points or subtrees rooted at the current node. Specifically, $B_i = [b_{i,l}, b_{i,r}]$ for $i \in [1, d]$ defines the closed interval in the i -th dimension, with $b_{i,l}$ and $b_{i,r}$ representing the lower and upper bounds, respectively. The component ptr is a set of pointers referencing the child nodes of the current internal node. In contrast, each leaf node stores the actual data points as d -dimensional vectors, denoted as $\mathbf{x} = (x_1, x_2, \dots, x_d)$.

Once the R-tree T is constructed, it can be used to efficiently perform range queries. Given a query request $\mathbf{q} = ([q_{1,l}, q_{1,r}], \dots, [q_{d,l}, q_{d,r}])$, we can perform a top-down search from the root node of T to find the required nodes.

- **Internal nodes:** To determine whether to traverse the child nodes of the current internal node, the algorithm checks if the node's MBR $\mathbf{B}$ intersects with the query range $\mathbf{q}$ across all dimensions, i.e., whether $[q_{i,l}, q_{i,r}] \cap B_i \neq \emptyset$ for all $i \in \{1, 2, \dots, d\}$. If the intersection holds, all child nodes need to be traversed; otherwise, they can be pruned.

- **Leaf nodes:** Each leaf node stores a d -dimensional data point $\mathbf{x} = (x_1, x_2, \dots, x_d)$. A point is included in the query result if $x_i \in [q_{i,l}, q_{i,r}]$ for all $i \in \{1, 2, \dots, d\}$.

By selectively traversing branches whose MBRs intersect the query range, the R-tree achieves efficient spatial pruning, significantly reducing the search space and improving query performance, as detailed in Algorithm 1.

B. Prefix Encoding

Prefix encoding is a data compression method that minimizes the bits needed to represent a set of strings by exploiting shared prefixes [23]. For instance, the binary strings

$\{0100, 0101, 0110\}$ can be compressed as $\{010*, 0110\}$, where the first two strings share the prefix 010, represented as 010*. Here, "*" denotes a wildcard, indicating either 0 or 1. This technique allows a range R to be expressed by a minimal prefix set $\mathcal{P}_R$, whose union exactly covers R . For example, the range $R = [2, 4]$ can be represented as $\{010, 011, 100\}$, with $\mathcal{P}_R = \{01*, 100\}$ as its minimal prefix set.

C. Three-Party Secret Sharing

The three-party secret sharing scheme, denoted by Π_{TPSS} , is realized across three cloud servers: CS_1 , CS_2 , and CS_3 [24]. We mainly focus on binary data sharing, so the data are shared over the ring $\mathbb{Z}_2$. It involves several algorithms, as shown below.

- $\Pi_{\text{TPSS}}.\text{Share}(x)$. Given a value $x \in \{0, 1\}$, the data sharing algorithm generates three random numbers $\{\alpha_{x,1}, \alpha_{x,2}, \beta_x\} \in \mathbb{Z}_2$ satisfying that $\alpha_{x,1} \oplus \alpha_{x,2} = \alpha_x$ and $x = \beta_x \oplus \alpha_x$. Then, $(\alpha_{x,1}, \alpha_{x,2})$ are distributed to CS_1 ; $(\alpha_{x,1}, \beta_x)$ are distributed to CS_2 ; and $(\alpha_{x,2}, \beta_x)$ are distributed to CS_3 . Let $\llbracket x \rrbracket = \{\text{CS}_1 : (\alpha_{x,1}, \alpha_{x,2}), \text{CS}_2 : (\alpha_{x,1}, \beta_x), \text{CS}_3 : (\alpha_{x,2}, \beta_x)\}$ denote the secret sharing form of x .

- $\Pi_{\text{TPSS}}.\text{XOR}(\llbracket x \rrbracket, \llbracket y \rrbracket)$. Given $\{\llbracket x \rrbracket, \llbracket y \rrbracket\}$, the three cloud servers can locally compute $\llbracket x \oplus y \rrbracket$; the computation process can be divided into an online phase and an offline phase.

- 1) **Offline phase:** CS_1 computes $\alpha_{z,1} = \alpha_{x,1} \oplus \alpha_{y,1}$, $\alpha_{z,2} = \alpha_{x,2} \oplus \alpha_{y,2}$; CS_2 computes $\alpha_{z,1} = \alpha_{x,1} \oplus \alpha_{y,1}$; and CS_3 computes $\alpha_{z,2} = \alpha_{x,2} \oplus \alpha_{y,2}$.
- 2) **Online phase:** CS_2 computes $\beta_z = \beta_x \oplus \beta_y$ and CS_3 computes $\beta_z = \beta_x \oplus \beta_y$.

Then, $\llbracket x \oplus y \rrbracket$ is shared in the form of $\{\text{CS}_1 : (\alpha_{z,1}, \alpha_{z,2}), \text{CS}_2 : (\alpha_{z,1}, \beta_z), \text{CS}_3 : (\alpha_{z,2}, \beta_z)\}$.

- $\Pi_{\text{TPSS}}.\text{Rec}(\llbracket x \rrbracket)$. The reconstruction protocol takes $\llbracket x \rrbracket$ as input and outputs the plaintext x . For any party CS_i with $i \in \{1, 2, 3\}$, the missing share needed for reconstruction is held by the other two parties. One of them sends the missing share to CS_i , which computes $x = \beta_x \oplus \alpha_{x,1} \oplus \alpha_{x,2}$.

- $\Pi_{\text{TPSS}}.\text{AND}(\llbracket x \rrbracket, \llbracket y \rrbracket)$. The protocol takes $\llbracket x \rrbracket$ and $\llbracket y \rrbracket$ as inputs and outputs $\llbracket z \rrbracket$, where $z = x \wedge y$. It consists of an offline phase and an online phase.

- 1) **Offline phase:** CS_1 and CS_2 sample two random numbers $\alpha_{z,1}$ and $\alpha_{xy,1}$. CS_1 and CS_3 randomly sample a random number $\alpha_{z,2}$. Meanwhile, CS_1 computes $\alpha_{xy} = \alpha_x \wedge \alpha_y$. Then, CS_1 sends $\alpha_{xy,2} = \alpha_{xy} \oplus \alpha_{xy,1}$ to CS_3 .
- 2) **Online phase:** CS_2 computes $\beta_{z,1} = (\beta_x \wedge \alpha_{y,1}) \oplus (\beta_y \wedge \alpha_{x,1}) \oplus \alpha_{z,1} \oplus \alpha_{xy,1}$; CS_3 computes $\beta_{z,2} = (\beta_x \wedge \beta_y) \oplus (\beta_x \wedge \alpha_{y,2}) \oplus (\beta_y \wedge \alpha_{x,2}) \oplus \alpha_{z,2} \oplus \alpha_{xy,2}$. Finally, CS_2 and CS_3 exchange $\beta_{z,1}$ and $\beta_{z,2}$ and locally reconstruct β_z as $\beta_z = \beta_{z,1} \oplus \beta_{z,2}$.

IV. OUR PROPOSED RANGE QUERY SCHEME

In this section, we first present prefix-encoding-based multi-dimensional range queries. Then, based on three-party secret sharing, we introduce an adaptive three-party secret sharing scheme. Subsequently, we present several private algorithms designed in our work, and finally construct our SLIMQ scheme based on these algorithms.

A. Prefix Encoding-Based Multi-Dimensional Range Query

In this section, we leverage the prefix encoding technique to improve the efficiency of R-tree-based multi-dimensional range queries. First, we show the representation of data in R-trees and query requests based on the prefix encoding technique. Then, we transform the processing of R-tree-based multi-dimensional range queries into a prefix matching problem.

• Representation of Data in R-tree and Query Requests:

First, all numerical values in the R-tree are uniformly represented in binary form. Consider an internal node $(\mathbf{B}, \text{ptr})$, where $\mathbf{B} = (B_1, B_2, \dots, B_d)$ and $B_i = [b_{i,l}, b_{i,r}]$ for each $i \in [1, d]$. We encode the endpoints of each interval as w -bit binary sequences, i.e., $b_{i,l} = b_{i,l}[1]b_{i,l}[2] \dots b_{i,l}[w]$ and $b_{i,r} = b_{i,r}[1]b_{i,r}[2] \dots b_{i,r}[w]$. For a leaf node $\mathbf{x} = (x_1, x_2, \dots, x_d)$, each dimension is similarly represented as a w -bit binary sequence, i.e., $x_i = x_i[1]x_i[2] \dots x_i[w]$, where $i \in [1, d]$. Subsequently, the query request $\mathbf{q} = ([q_{1,l}, q_{1,r}], \dots, [q_{d,l}, q_{d,r}])$ is encoded. For each dimensional query interval, it is represented as $[q_{i,l}, q_{i,r}] = (\mathcal{P}_{[0, q_{i,l}-1]}, \mathcal{P}_{[0, q_{i,r}]}, \mathcal{P}_{[q_{i,l}, q_{i,r}]})$, where $i \in [1, d]$. For a given interval $[l, r]$, we use $\mathcal{P}_{[l, r]}$ to denote its minimal prefix set. This means that $\mathcal{P}_{[l, r]}$ matches the binary string representations of all integers in the interval $[l, r]$. Thus, for a given integer x , we have $x \in [l, r] \Leftrightarrow x$ matches $\mathcal{P}_{[l, r]}$.

• *Multi-dimensional Range Query Transformation:* In the R-tree-based range query process, two key operations are required: (1) point-range intersection test, i.e., determining whether x_i lies within the interval $[q_{i,l}, q_{i,r}]$; and (2) range intersection test, i.e., determining whether the interval $[q_{i,l}, q_{i,r}]$ intersects with $[b_{i,l}, b_{i,r}]$. For the point-range intersection test, we have $x_i \in [q_{i,l}, q_{i,r}]$ if and only if x_i matches at least one prefix in $\mathcal{P}_{[q_{i,l}, q_{i,r}]}$. For the range intersection test, it is well known that two intervals intersect if and only if $[q_{i,l}, q_{i,r}] \cap [b_{i,l}, b_{i,r}] \neq \emptyset$. This condition is equivalent to $(b_{i,l} \leq q_{i,r}) \wedge (b_{i,r} \geq q_{i,l})$. Under this condition, the intervals $[0, q_{i,r}]$ and $[0, q_{i,l}-1]$ can be represented by their minimal prefix sets $\mathcal{P}_{[0, q_{i,r}]}$ and $\mathcal{P}_{[0, q_{i,l}-1]}$, respectively. Consequently, the range intersection test can be reduced to prefix matching checks on the endpoints: verify whether $b_{i,l}$ matches $\mathcal{P}_{[0, q_{i,r}]}$ and whether $b_{i,r}$ does *not* match $\mathcal{P}_{[0, q_{i,l}-1]}$.

$$\begin{aligned} & [q_{i,l}, q_{i,r}] \cap [b_{i,l}, b_{i,r}] \neq \emptyset \\ \Leftrightarrow & \text{Match}(b_{i,l}, \mathcal{P}_{[0, q_{i,r}]}) \wedge \neg \text{Match}(b_{i,r}, \mathcal{P}_{[0, q_{i,l}-1]}) \quad (1) \end{aligned}$$

Lemma 1: For any valid intervals $Q_i = [q_{i,l}, q_{i,r}]$ and $B_i = [b_{i,l}, b_{i,r}]$ over the w -bit domain, the above range-intersection transformation is correct. That is, $Q_i \cap B_i \neq \emptyset$ if and only if $b_{i,l}$ matches at least one prefix in $\mathcal{P}_{[0, q_{i,r}]}$ and $b_{i,r}$ does not match any prefix in $\mathcal{P}_{[0, q_{i,l}-1]}$.

Proof: Since Q_i and B_i are valid intervals, we have $q_{i,l} \leq q_{i,r}$ and $b_{i,l} \leq b_{i,r}$. If $Q_i \cap B_i \neq \emptyset$, there exists a value z such that $q_{i,l} \leq z \leq q_{i,r}$ and $b_{i,l} \leq z \leq b_{i,r}$. Hence, $b_{i,l} \leq q_{i,r}$ and $b_{i,r} \geq q_{i,l}$. Conversely, if $b_{i,l} \leq q_{i,r}$ and $b_{i,r} \geq q_{i,l}$, then $\max\{q_{i,l}, b_{i,l}\} \leq \min\{q_{i,r}, b_{i,r}\}$, so the two intervals share at least one value and thus $Q_i \cap B_i \neq \emptyset$. Therefore,

$$Q_i \cap B_i \neq \emptyset \Leftrightarrow (b_{i,l} \leq q_{i,r}) \wedge (b_{i,r} \geq q_{i,l}). \quad (2)$$

The condition $b_{i,l} \leq q_{i,r}$ is equivalent to $b_{i,l} \in [0, q_{i,r}]$, and the condition $b_{i,r} \geq q_{i,l}$ is equivalent to $b_{i,r} \notin [0, q_{i,l}-1]$. By the definition of minimal prefix cover, $\mathcal{P}_{[l, r]}$ is a set of disjoint prefix-defined subranges whose union is exactly $[l, r]$. Thus, a w -bit value x belongs to $[l, r]$ if and only if it matches at least one prefix in $\mathcal{P}_{[l, r]}$. Applying this property to $[0, q_{i,r}]$ and $[0, q_{i,l}-1]$ proves the correctness of the prefix-based range-intersection transformation. Since Π_{PMatch} evaluates the same prefix-matching predicate over secret shares, the encrypted checks used in Π_{RInter} preserve the above Boolean relation.

B. Adaptive Three-Party Secret Sharing

In this section, we introduce a query-adaptive three-party secret sharing scheme, denoted by Π_{aTPSS} , which differs from Π_{TPSS} in its data-sharing and multiplication algorithms, as shown below.

• $\Pi_{\text{aTPSS}}.\text{Share}(x)$. This protocol is used to share the data owner's data among the servers. Let $F(\cdot, \cdot) : \{0, 1\}^\kappa \times \{0, 1\}^* \rightarrow \mathbb{Z}_2$ be a PRF. Suppose that the data owner has a common key $k_{1,2}$ with $\{\text{CS}_1, \text{CS}_2\}$, has a common key $k_{1,3}$ with $\{\text{CS}_1, \text{CS}_3\}$, and has a common key $k_{2,3}$ with $\{\text{CS}_2, \text{CS}_3\}$. The adaptive sharing protocol makes full use of non-interactive random-value generation via the PRF and the high-speed communication link between servers to reduce the communication overhead on the data owner side. Detailed steps are shown below.

Step 1: The data owner collaborates with $\{\text{CS}_1, \text{CS}_2\}$ to jointly generate random values $\alpha_{x,1} \in \mathbb{Z}_2$ using $F_{k_{1,2}}(\text{counter})$, where *counter* is an accumulator kept by the data owner and $\{\text{CS}_1, \text{CS}_2\}$. Similarly, the data owner collaborates with $\{\text{CS}_1, \text{CS}_3\}$ to jointly generate random values $\alpha_{x,2} \in \mathbb{Z}_2$ using $F_{k_{1,3}}(\text{counter})$.

Step 2: The data owner computes $\beta_x = x \oplus \alpha_{x,1} \oplus \alpha_{x,2}$ and sends β_x to CS_2 .

Step 3: Upon receiving β_x , CS_2 forwards it to CS_3 .

In this case, the data owner only needs to transmit one bit per shared data bit.

• $\Pi_{\text{aTPSS}}.\text{AND}([x], [y])$. This protocol takes $[x]$ and $[y]$ as inputs and outputs $[z]$, where $z = x \wedge y$. The protocol is adaptively designed for the $\wedge$ operation in outsourced query services. In the outsourced query setting, the data owner secret-shares its data among the servers in advance. Thus, for a given $[x]$, the servers collectively hold the corresponding shares of $\alpha_{x,1}$, $\alpha_{x,2}$, and β_x . In contrast, for a query value y , the servers can precompute only $\alpha_{y,1}$ and $\alpha_{y,2}$, while β_y is unavailable until the query request arrives. As a result, we design an adaptive protocol for such $[x]$ and $[y]$. The protocol consists of an offline phase and an online phase:

- 1) *Offline phase:* In addition to executing the offline phase of the base protocol $\Pi_{\text{TPSS}}.\text{AND}([x], [y])$, CS_2 computes $\beta_{z,1} = (\beta_x \wedge \alpha_{y,1}) \oplus \alpha_{x,1} \oplus \alpha_{xy,1}$. Meanwhile, CS_3 computes $\beta_{z,2} = (\beta_x \wedge \alpha_{y,2}) \oplus \alpha_{x,2} \oplus \alpha_{xy,2}$.
- 2) *Online phase:* Upon receiving the query input, CS_2 updates its local value to $\beta_{z,1} = \beta_{z,1} \oplus (\beta_y \wedge \alpha_{x,1})$; CS_3 updates its local value to $\beta_{z,2} = \beta_{z,2} \oplus (\beta_y \wedge \alpha_{x,2})$. Subsequently, CS_2 and CS_3 exchange the updated

$\beta_{z,1}$ and $\beta_{z,2}$, and locally reconstruct β_z as $\beta_z = \beta_{z,1} \oplus \beta_{z,2}$.

C. Private Algorithms

In this section, we propose a three-party secret sharing based private prefix matching algorithm and private point/range intersection test algorithms.

• *Private Prefix Matching Algorithm*: Let x be a w -bit number. Let $p = p[1]p[2] \cdots p[w]$ be a w -bit prefix, such as 001*. The private prefix matching algorithm privately determines whether x matches p and consists of three sub-algorithms.

1) $\Pi_{\text{PMatch.Enc}}(x)$: The w -bit number x is shared through the following steps.

Step 1: Represent x in binary form as $x = x[1]x[2] \cdots x[w]$, where $x[1]$ is the most significant bit of x .

Step 2: Share x among servers as $\llbracket x \rrbracket = \llbracket x[1] \rrbracket \llbracket x[2] \rrbracket \cdots \llbracket x[w] \rrbracket$. Finally, the algorithm returns $\llbracket x \rrbracket$.

2) $\Pi_{\text{PMatch.TokenGen}}(p)$: The w -bit prefix $p = p[1]p[2] \cdots p[w]$ is shared as the following steps.

Step 1: Represent p to $p' = p'[1]p'[2] \cdots p'[w]$ and $f = f[1]f[2] \cdots f[w]$, where

$$p'[j] = \begin{cases} p[j], & p[j] \neq "*" \\ 0, & p[j] = "*" \end{cases} \quad (3)$$

$$f[j] = \begin{cases} 1, & p[j] \neq "*" \\ 0, & p[j] = "*" \end{cases} \quad (4)$$

Step 2: Share p' and f among servers as

$$\begin{cases} \llbracket p' \rrbracket = \llbracket p'[1] \rrbracket \llbracket p'[2] \rrbracket \cdots \llbracket p'[w] \rrbracket \\ \llbracket f \rrbracket = \llbracket f[1] \rrbracket \llbracket f[2] \rrbracket \cdots \llbracket f[w] \rrbracket \end{cases} \quad (5)$$

3) $\Pi_{\text{PMatch.Query}}(\llbracket x \rrbracket, \llbracket p' \rrbracket, \llbracket f \rrbracket)$: This algorithm takes $\{\llbracket x \rrbracket, \llbracket p' \rrbracket, \llbracket f \rrbracket\}$ as inputs and returns whether x matches p . Detailed steps are shown below.

Step 1: Servers compute $\llbracket z[j] \rrbracket = 1 \oplus (\llbracket x[j] \rrbracket \oplus \llbracket p'[j] \rrbracket) \wedge \llbracket f[j] \rrbracket$ for $1 \leq j \leq w$.

Step 2: Servers compute $\llbracket \gamma \rrbracket = 1 \oplus (\wedge_{j=1}^w \llbracket z[j] \rrbracket)$, where “ $\gamma = 0$ ” denotes that x matches p , and “ $\gamma = 1$ ” denotes that x does not match p .

• *Private Point Intersection Test Algorithm*: Let $\mathbf{x} = (x_1, x_2, \dots, x_d)$ be a record and $\mathbf{q} = ([q_{1,l}, q_{1,r}], \dots, [q_{d,l}, q_{d,r}])$ be a query range. The private point intersection test algorithm Π_{PInter} privately determines whether $\mathbf{x} \in \mathbf{q}$ and consists of three sub-algorithms.

1) $\Pi_{\text{PInter.Enc}}(\mathbf{x})$: Each x_i is shared as $\llbracket x_i \rrbracket = (\llbracket x_i[1] \rrbracket, \llbracket x_i[2] \rrbracket, \dots, \llbracket x_i[w] \rrbracket)$, where $\llbracket x_i \rrbracket = \Pi_{\text{PMatch.Enc}}(x_i)$ for $1 \leq i \leq d$. Then, let $\llbracket \mathbf{x} \rrbracket = (\llbracket x_1 \rrbracket, \llbracket x_2 \rrbracket, \dots, \llbracket x_d \rrbracket)$.

2) $\Pi_{\text{PInter.TokenGen}}(\mathbf{q})$: For $1 \leq i \leq d$, each interval $[q_{i,l}, q_{i,r}]$ is processed as follows.

Step 1: Convert $[q_{i,l}, q_{i,r}]$ into a prefix set $\mathcal{P}_{[q_{i,l}, q_{i,r}]} = \{p_{i,1}, p_{i,2}, \dots\}$.

Step 2: Generate a query token for each $p_{i,j} \in \mathcal{P}_{[q_{i,l}, q_{i,r}]}$ by computing $\{\llbracket p'_{i,j} \rrbracket, \llbracket f_{i,j} \rrbracket\} = \Pi_{\text{PMatch.TokenGen}}(p_{i,j})$. Then, let $\llbracket \mathbf{q} \rrbracket^{\text{PInter}} = \{\llbracket p'_{i,j} \rrbracket, \llbracket f_{i,j} \rrbracket\}_{j=1}^{|\mathcal{P}_{[q_{i,l}, q_{i,r}]}|}$.

3) $\Pi_{\text{PInter.Query}}(\llbracket \mathbf{x} \rrbracket, \llbracket \mathbf{q} \rrbracket^{\text{PInter}})$: The algorithm takes $\{\llbracket \mathbf{x} \rrbracket, \llbracket \mathbf{q} \rrbracket^{\text{PInter}}\}$ as inputs and outputs whether $\mathbf{x} \in \mathbf{q}$ as follows.

Step 1: For each $\llbracket x_i \rrbracket$ and each $\{\llbracket p'_{i,j} \rrbracket, \llbracket f_{i,j} \rrbracket\} \in \llbracket \mathcal{P}_{[q_{i,l}, q_{i,r}]} \rrbracket$, servers compute $\llbracket \gamma_{i,j} \rrbracket = \Pi_{\text{PMatch.Query}}(\llbracket x_i \rrbracket, \llbracket p'_{i,j} \rrbracket, \llbracket f_{i,j} \rrbracket)$ for $1 \leq j \leq |\mathcal{P}_{[q_{i,l}, q_{i,r}]}|$.

Step 2: Servers compute $\llbracket \gamma_i \rrbracket = \wedge_{j=1}^{|\mathcal{P}_{[q_{i,l}, q_{i,r}]}|} \llbracket \gamma_{i,j} \rrbracket$.

Step 3: Servers compute $\llbracket \gamma \rrbracket = \wedge_{i=1}^d (1 \oplus \llbracket \gamma_i \rrbracket)$ and recover γ by computing $\gamma = \Pi_{\text{TPSS.Rec}}(\llbracket \gamma \rrbracket)$. If $\gamma = 1$, this indicates that $\mathbf{x} \in \mathbf{q}$; otherwise, $\mathbf{x} \notin \mathbf{q}$.

• *Private Range Intersection Test Algorithm*: Let $\mathbf{B} = (B_1, B_2, \dots, B_d)$ be an MBR and $\mathbf{q} = ([q_{1,l}, q_{1,r}], \dots, [q_{d,l}, q_{d,r}])$ be a query range, where $B_i = [b_{i,l}, b_{i,r}]$ for $1 \leq i \leq d$. The private range intersection test algorithm privately determines whether $\mathbf{B} \cap \mathbf{q} = \emptyset$ and consists of three sub-algorithms.

1) $\Pi_{\text{RInter.Enc}}(\mathbf{B})$: For each range $[b_{i,l}, b_{i,r}]$, it is shared as $\llbracket [b_{i,l}, b_{i,r}] \rrbracket$, where $\llbracket b_{i,l} \rrbracket = \Pi_{\text{PMatch.Enc}}(b_{i,l})$ and $\llbracket b_{i,r} \rrbracket = \Pi_{\text{PMatch.Enc}}(b_{i,r})$. Then, let $\llbracket \mathbf{B} \rrbracket = (\llbracket [b_{1,l}, b_{1,r}] \rrbracket, \llbracket [b_{2,l}, b_{2,r}] \rrbracket, \dots, \llbracket [b_{d,l}, b_{d,r}] \rrbracket)$.

2) $\Pi_{\text{RInter.TokenGen}}(\mathbf{q})$: For $1 \leq i \leq d$, each interval $[q_{i,l}, q_{i,r}]$ is processed as follows.

Step 1: Construct two prefix sets $\mathcal{P}_{[0, q_{i,l}-1]} = \{p_{i,l,1}, p_{i,l,2}, \dots\}$ and $\mathcal{P}_{[0, q_{i,r}]} = \{p_{i,r,1}, p_{i,r,2}, \dots\}$.

Step 2: Generate a query token for each $p_{i,l,j} \in \mathcal{P}_{[0, q_{i,l}-1]}$ by computing $\{\llbracket p'_{i,l,j} \rrbracket, \llbracket f_{i,l,j} \rrbracket\} = \Pi_{\text{PMatch.TokenGen}}(p_{i,l,j})$. Meanwhile, generate a query token for each $p_{i,r,j} \in \mathcal{P}_{[0, q_{i,r}]}$ by computing $\{\llbracket p'_{i,r,j} \rrbracket, \llbracket f_{i,r,j} \rrbracket\} = \Pi_{\text{PMatch.TokenGen}}(p_{i,r,j})$.

Step 3: Let $\llbracket \mathbf{q} \rrbracket^{\text{RInter}} = \{\llbracket p'_{i,l,j} \rrbracket, \llbracket f_{i,l,j} \rrbracket\}_{j=1}^{|\mathcal{P}_{[0, q_{i,l}-1]}|} \cup \{\llbracket p'_{i,r,j} \rrbracket, \llbracket f_{i,r,j} \rrbracket\}_{j=1}^{|\mathcal{P}_{[0, q_{i,r}]}|}$.

3) $\Pi_{\text{RInter.Query}}(\llbracket \mathbf{B} \rrbracket, \llbracket \mathbf{q} \rrbracket^{\text{RInter}})$: The algorithm takes $\{\llbracket \mathbf{B} \rrbracket, \llbracket \mathbf{q} \rrbracket^{\text{RInter}}\}$ as inputs and outputs whether $\mathbf{B} \cap \mathbf{q} = \emptyset$ as follows.

Step 1: For each pair $\llbracket b_{i,l} \rrbracket$ and each $\{\llbracket p'_{i,r,j} \rrbracket, \llbracket f_{i,r,j} \rrbracket\} \in \llbracket \mathcal{P}_{[0, q_{i,r}]} \rrbracket$, servers compute

$$\llbracket \gamma_{i,l,j} \rrbracket = \Pi_{\text{PMatch.Query}}(\llbracket b_{i,l} \rrbracket, \llbracket p'_{i,r,j} \rrbracket, \llbracket f_{i,r,j} \rrbracket) \quad (6)$$

for $1 \leq i \leq d$ and $1 \leq j \leq |\mathcal{P}_{[0, q_{i,r}]}|$.

Step 2: For each pair $\llbracket b_{i,r} \rrbracket$ and each $\{\llbracket p'_{i,l,j} \rrbracket, \llbracket f_{i,l,j} \rrbracket\} \in \llbracket \mathcal{P}_{[0, q_{i,l}-1]} \rrbracket$, servers compute

$$\llbracket \gamma_{i,r,j} \rrbracket = \Pi_{\text{PMatch.Query}}(\llbracket b_{i,r} \rrbracket, \llbracket p'_{i,l,j} \rrbracket, \llbracket f_{i,l,j} \rrbracket) \quad (7)$$

for $1 \leq i \leq d$ and $1 \leq j \leq |\mathcal{P}_{[0, q_{i,l}-1]}|$.

Step 3: Servers compute

$$\begin{cases} \llbracket \gamma_{i,l} \rrbracket = \wedge_{j=1}^{|\mathcal{P}_{[0, q_{i,r}]}|} \llbracket \gamma_{i,l,j} \rrbracket \\ \llbracket \gamma_{i,r} \rrbracket = \wedge_{j=1}^{|\mathcal{P}_{[0, q_{i,l}-1]}|} \llbracket \gamma_{i,r,j} \rrbracket \end{cases} \quad (8)$$

for $1 \leq i \leq d$.

Step 4: Servers compute $\llbracket \gamma_i \rrbracket = \llbracket \gamma_{i,l} \rrbracket \oplus \llbracket \gamma_{i,r} \rrbracket$ for $1 \leq i \leq d$.

Step 5: Servers compute $\llbracket \gamma \rrbracket = \wedge_{i=1}^d \llbracket \gamma_i \rrbracket$ and recover γ by computing $\gamma = \Pi_{\text{TPSS.Rec}}(\llbracket \gamma \rrbracket)$. If $\gamma = 1$, it denotes that $\mathbf{B} \cap \mathbf{q} \neq \emptyset$. Otherwise, $\mathbf{B} \cap \mathbf{q} = \emptyset$.

D. Our Proposed Scheme

Our privacy-preserving range query scheme consists of four main components: Setup, Data Outsourcing, Query Token Generation, and Query Processing.

• *Setup*: Given a security parameter κ , the setup phase proceeds as follows.

Step 1: The data owner chooses a PRF $F(\cdot, \cdot) : \{0, 1\}^\kappa \times \{0, 1\}^* \rightarrow \mathbb{Z}_2$ and four random numbers $\{k, k_{1,2}, k_{1,3}, k_{2,3}\} \in \{0, 1\}^\kappa$ as the keys of PRF.

Step 2: The data owner sends $\{F(\cdot, \cdot), k, k_{1,2}, k_{1,3}\}$ to CS_1 ; sends $\{F(\cdot, \cdot), k, k_{1,2}, k_{2,3}\}$ to CS_2 ; and sends $\{F(\cdot, \cdot), k, k_{1,3}, k_{2,3}\}$ to CS_3 .

• *Data Outsourcing*: The data owner outsources its dataset $\mathcal{X} = \{\mathbf{x}_j = (x_{j,1}, x_{j,2}, \dots, x_{j,d})\}_{j=1}^n$ as follows.

Step 1: The data owner constructs an R-tree T based on the plaintext dataset $\mathcal{X}$.

Step 2: The data owner shares T with servers. Before sharing the tree, the data owner fixes the bit length w as a global system parameter, which defines the domain $\mathcal{D}_w = \{0, \dots, 2^w - 1\}$. In practice, w is selected according to the public application domain or a conservative upper bound, and all outsourced values and valid query bounds should lie in $\mathcal{D}_w$. Then, internal nodes and leaf nodes of T are shared as follows.

1) For each internal node with $(\mathbf{B}, \text{ptr})$, the data owner shares it as $[\mathbf{B}] = \Pi_{\text{RInter.Enc}}(\mathbf{B})$.

2) For each leaf node with $\mathbf{x}$, the data owner shares it as $[\mathbf{x}] = \Pi_{\text{PInter.Enc}}(\mathbf{x})$.

Finally, the tree T is shared in the form of $[T]$.

• *Query Token Generation*: Given a query range $\mathbf{q}$ whose intervals are within $\mathcal{D}_w$, the query user generates two query tokens as $[\mathbf{q}]^{\text{PInter}} = \Pi_{\text{PInter.TokenGen}}(\mathbf{q})$ and $[\mathbf{q}]^{\text{RInter}} = \Pi_{\text{RInter.TokenGen}}(\mathbf{q})$, which are shared among servers.

• *Query Processing*: On inputs $\{[T], [\mathbf{q}]^{\text{PInter}}, [\mathbf{q}]^{\text{RInter}}\}$, servers process the query as follows.

Step 1: Traverse the tree $[T]$ to obtain the query result.

1) For each internal node with $\{[\mathbf{B}], \text{ptr}\}$, servers first compute $\gamma = \Pi_{\text{RInter.Query}}([\mathbf{B}], [\mathbf{q}]^{\text{RInter}})$. If $\gamma = 1$, it denotes that $\mathbf{B} \cap \mathbf{q} \neq \emptyset$. In this case, servers continue to search the child nodes of the current node. Otherwise, the child nodes of the current node are pruned.

2) For each leaf node with $[\mathbf{x}]$, servers first compute $\gamma = \Pi_{\text{PInter.Query}}([\mathbf{x}], [\mathbf{q}]^{\text{PInter}})$. If $\gamma = 1$, it denotes that $\mathbf{x} \in \mathbf{q}$. In this case, servers put $[\mathbf{x}]$ into the query result, i.e., $\mathcal{R} = \mathcal{R} \cup \{[\mathbf{x}]\}$.

After this step, servers obtain the query result $\mathcal{R}$.

Step 2: For each $[\mathbf{x}] \in \mathcal{R}$, CS_1 returns $(\alpha_{x_1}, \alpha_{x_2}, \dots, \alpha_{x_d})$ to the query user, and CS_2 returns $(\beta_{x_1}, \beta_{x_2}, \dots, \beta_{x_d})$ to the query user, where $\alpha_{x_i} = \alpha_{x_i,1} \oplus \alpha_{x_i,2}$ for $1 \leq i \leq d$. Then, the query user can recover $\mathbf{x} = (x_1, x_2, \dots, x_d)$ by computing $x_i = \alpha_{x_i} \oplus \beta_{x_i}$. The detailed ciphertext-domain query processing procedure is summarized in Algorithm 2.

V. SECURITY ANALYSIS

In this section, we formally prove the security of our SLIMQ scheme under the simulation-based real and ideal worlds security model [28]. Since the Π_{aTPSS} scheme, the Π_{PMatch} algorithm, the Π_{PInter} algorithm, and the Π_{RInter} algorithm are key building

Algorithm 2: Secure Range Query over Encrypted R-tree.

Input: Encrypted R-tree $[T]$, query tokens $[\mathbf{q}]^{\text{PInter}}, [\mathbf{q}]^{\text{RInter}}$

Output: Set of plaintext points $\mathcal{A}$ within the query range $\mathbf{q}$

Initialize the plaintext result set $\mathcal{A} \leftarrow \emptyset$;

Initialize the encrypted result set $\mathcal{R} \leftarrow \emptyset$;

Initialize an empty stack $S \leftarrow \emptyset$;

$S.\text{push}([T].\text{root})$;

while S is not empty **do**

 Pop a node $Node$ from S ;

if $Node$ is an internal node with $([\mathbf{B}], \text{ptr})$ **then**

 Servers jointly compute

$\gamma = \Pi_{\text{RInter.Query}}([\mathbf{B}], [\mathbf{q}]^{\text{RInter}})$;

if $\gamma = 1$ (i.e., $\mathbf{B} \cap \mathbf{q} \neq \emptyset$) **then**

 Push all child nodes of $Node$ to S ;

else

 Prune all child nodes of $Node$;

else if $Node$ is a leaf node with encrypted point $[\mathbf{x}]$ **then**

 Servers jointly compute

$\gamma = \Pi_{\text{PInter.Query}}([\mathbf{x}], [\mathbf{q}]^{\text{PInter}})$;

if $\gamma = 1$ (i.e., $\mathbf{x} \in \mathbf{q}$) **then**

$\mathcal{R} \leftarrow \mathcal{R} \cup \{[\mathbf{x}]\}$;

foreach $[\mathbf{x}] \in \mathcal{R}$ **do**

CS_1 locally computes $\alpha_{x_i} = \alpha_{x_i,1} \oplus \alpha_{x_i,2}$ for all $i \in \{1, \dots, d\}$ and sends $(\alpha_{x_1}, \alpha_{x_2}, \dots, \alpha_{x_d})$ to the query user;

CS_2 sends $(\beta_{x_1}, \beta_{x_2}, \dots, \beta_{x_d})$ to the query user;

 The query user locally recovers

$\mathbf{x} = (x_1, x_2, \dots, x_d)$ by computing

$x_i = \alpha_{x_i} \oplus \beta_{x_i}$ for all $i \in \{1, \dots, d\}$ and updates

$\mathcal{A} \leftarrow \mathcal{A} \cup \{\mathbf{x}\}$;

return $\mathcal{A}$;

blocks of our SLIMQ scheme, we first prove the security of these algorithms and then prove the security of our SLIMQ scheme. In the security model, there are three non-colluding semi-honest cloud servers $\{CS_i\}_{i=1}^3$ and three simulators $\{\mathcal{S}_i\}_{i=1}^3$, where $\mathcal{S}_i$ is responsible for simulating the view of CS_i in the ideal world for $1 \leq i \leq 3$. We prove the security of our scheme and foundational algorithms by showing the indistinguishability of CS_i 's views in the real and ideal worlds for $1 \leq i \leq 3$. Specifically, for each building block, we first specify its leakage, then construct the corresponding simulated views using only this leakage and the simulators of the underlying secret-sharing operations. For Π_{PMatch} , the leakage contains only the public ring and bit length w ; for Π_{PInter} and Π_{RInter} , it additionally includes the prefix-set sizes and the final intersection result. For the complete SLIMQ scheme, the leakage further includes the public parameters, the encrypted R-tree structure, the visited-node/access pattern, and the final encrypted result set. Under these leakage functions, the security of SLIMQ follows by sequentially composing the simulators of Π_{PMatch} , Π_{PInter} , and Π_{RInter} .

A. Security of Π_{aTPSS}

The Π_{aTPSS} scheme has two different algorithms compared with the TPSS scheme, including $\Pi_{\text{aTPSS.Share}}(x)$ and $\Pi_{\text{aTPSS.AND}}(\llbracket x \rrbracket, \llbracket y \rrbracket)$. In both algorithms, the leakage is only the ring $\mathbb{Z}_2$. In the following, we show the simulation of ideal worlds for them.

1) *Simulation of $\Pi_{\text{aTPSS.Share}}(x)$* : In this algorithm, CS_2 and CS_3 receive β_x . S_2 can choose a random $\beta_x^{2,\text{sim}} \in \mathbb{Z}_2$ as the simulation of β_x received by CS_2 . S_3 can also choose a random $\beta_x^{3,\text{sim}} \in \mathbb{Z}_2$ as the simulation of β_x received by CS_3 .

2) *Simulation of $\Pi_{\text{aTPSS.AND}}(\llbracket x \rrbracket, \llbracket y \rrbracket)$* : In this algorithm, CS_2 receives $\beta_{x,2}$, and CS_3 receives $\{\alpha_{xy,2}, \beta_{z,1}\}$. S_2 can choose a random $\beta_{x,2}^{2,\text{sim}} \in \mathbb{Z}_2$ as the simulation of $\beta_{x,2}$ received by CS_2 . S_3 can choose random numbers $\alpha_{xy,2}^{3,\text{sim}} \in \mathbb{Z}_2$ and $\beta_{z,1}^{3,\text{sim}} \in \mathbb{Z}_2$ as the simulation of $\{\alpha_{xy,2}, \beta_{z,1}\}$ received by CS_3 .

Definition 1. (Security of Π_{aTPSS}): The Π_{aTPSS} scheme is semantically secure *iff* there exist three simulators $\{\mathcal{S}_i\}_{i=1}^3$ that can simulate the ideal worlds of $\{\text{CS}_i\}_{i=1}^3$ based on the leakage $\mathbb{Z}_2$, and the probability that $\{\text{CS}_i\}_{i=1}^3$ can distinguish their views in the real and ideal worlds is negligible.

Proof: We prove the security of Π_{aTPSS} by showing that $\{\text{CS}_i\}_{i=1}^3$ has only a negligible probability of distinguishing their views in the real and ideal worlds. In the $\Pi_{\text{aTPSS.Share}}(x)$ algorithm, CS_2 can view $\{\alpha_{x,1}, \beta_x\}$ in the real world and $\{\alpha_{x,1}, \beta_x^{2,\text{sim}}\}$ in the ideal world. Since $x = \alpha_{x,1} \oplus \alpha_{x,2} \oplus \beta_x$ and CS_2 has no knowledge of $\alpha_{x,2}$, both $\{\alpha_{x,1}, \beta_x\}$ and $\{\alpha_{x,1}, \beta_x^{2,\text{sim}}\}$ are random, and CS_2 has only a negligible probability of distinguishing them. Similarly, CS_3 can view $\{\alpha_{x,2}, \beta_x\}$ in the real world and $\{\alpha_{x,2}, \beta_x^{3,\text{sim}}\}$ in the ideal world, and also has only a negligible probability of distinguishing them. In the $\Pi_{\text{aTPSS.AND}}(\llbracket x \rrbracket, \llbracket y \rrbracket)$ algorithm, CS_2 can view $\{\alpha_{x,1}, \alpha_{y,1}, \alpha_{xy,1}, \beta_{z,1}, \beta_{z,2}\}$ in the real world and $\{\alpha_{x,1}, \alpha_{y,1}, \alpha_{xy,1}, \beta_{z,1}, \beta_{z,2}^{2,\text{sim}}\}$ in the ideal world. Since $x = \alpha_{x,1} \oplus \alpha_{x,2} \oplus \beta_x$, $y = \alpha_{y,1} \oplus \alpha_{y,2} \oplus \beta_y$, $z = xy = \alpha_{z,1} \oplus \alpha_{z,2} \oplus \beta_z$, and CS_2 has no knowledge of $\{\alpha_{x,2}, \alpha_{y,2}, \alpha_{xy,2}, \alpha_{z,2}\}$, the probability that CS_2 can distinguish them is negligible. Similarly, the probability that CS_3 can distinguish its view in the real and ideal worlds is also negligible. $\square$

B. Security of Π_{PMatch} , Π_{PInter} , and Π_{RInter}

1) *Security of Π_{PMatch}* : The Π_{PMatch} algorithm publicly leaks the ring $\mathbb{Z}_2$, the parameter w , denoted by $\mathcal{L}_{\text{PMatch}}^{\text{pub}} = \{\mathbb{Z}_2, w\}$. Since the $\Pi_{\text{PMatch.Enc}}$ and $\Pi_{\text{PMatch.TokenGen}}$ algorithms share w -bit $\{x, p', f\}$ by calling $\Pi_{\text{aTPSS.Share}}$ w times and the security of $\Pi_{\text{aTPSS.Share}}$ has been proved, the $\Pi_{\text{PMatch.Enc}}$ and $\Pi_{\text{PMatch.TokenGen}}$ algorithms are semantically secure. On the other hand, since the $\Pi_{\text{PMatch.Query}}$ algorithm performs computations by calling $\Pi_{\text{TPSS.XOR}}$ and $\Pi_{\text{aTPSS.AND}}$ algorithms and the security of $\Pi_{\text{TPSS.XOR}}$ and $\Pi_{\text{aTPSS.AND}}$ has been proved, the $\Pi_{\text{PMatch.Query}}$ algorithm is semantically secure.

2) *Security of Π_{PInter} and Π_{RInter}* : The Π_{PInter} algorithm publicly leaks the ring $\mathbb{Z}_2$, the parameter w , and the dimension d , denoted by $\mathcal{L}_{\text{PInter}}^{\text{pub}} = \{\mathbb{Z}_2, w, d\}$. The $\Pi_{\text{PInter.TokenGen}}(\mathbf{q})$ additionally leaks $\{|\mathcal{P}_{[q_{i,l}, q_{i,r}]}\}|_{i=1}^d$,

denoted by $\mathcal{L}_{\text{PInter.TokenGen}}(\mathbf{q}) = \{|\mathcal{P}_{[q_{i,l}, q_{i,r}]}\}|_{i=1}^d$. The $\Pi_{\text{PInter.Query}}(\llbracket \mathbf{x} \rrbracket, \llbracket \mathbf{q} \rrbracket^{\text{PInter}})$ additionally leaks the result γ , denoted by $\mathcal{L}_{\text{PInter.Query}}(\llbracket \mathbf{x} \rrbracket, \llbracket \mathbf{q} \rrbracket^{\text{PInter}}) = \{\gamma\}$. Based on $\{\mathcal{L}_{\text{PInter}}^{\text{pub}}, \mathcal{L}_{\text{PInter.TokenGen}}(\mathbf{q}), \mathcal{L}_{\text{PInter.Query}}(\llbracket \mathbf{x} \rrbracket, \llbracket \mathbf{q} \rrbracket^{\text{PInter}})\}$, we can simulate the ideal world of Π_{PInter} . First, since the $\Pi_{\text{PInter.Enc}}(\mathbf{x})$ algorithm shares d w -bit $\{x_i\}_{i=1}^d$ by calling $\Pi_{\text{PMatch.Enc}}$, its simulation can be conducted using the simulation of $\Pi_{\text{PMatch.Enc}}$. Second, since the $\Pi_{\text{PInter.TokenGen}}(\mathbf{q})$ algorithm generates the token for each $p_{i,j} \in \mathcal{P}_{[q_{i,l}, q_{i,r}]}$ using $\Pi_{\text{PMatch.TokenGen}}$, its simulation can be conducted using the simulation of $\Pi_{\text{PMatch.TokenGen}}$. Third, $\Pi_{\text{PInter.Query}}(\llbracket \mathbf{x} \rrbracket, \llbracket \mathbf{q} \rrbracket^{\text{PInter}})$ performs computations using the $\Pi_{\text{PMatch.Query}}$, $\Pi_{\text{TPSS.XOR}}$, and $\Pi_{\text{aTPSS.AND}}$ algorithms, and its simulation can be conducted using the simulations of these algorithms. Since the probability that $\{\text{CS}_i\}_{i=1}^3$ can distinguish their views in the real and ideal worlds for the $\Pi_{\text{PMatch.Enc}}$, $\Pi_{\text{PInter.TokenGen}}(\mathbf{q})$, $\Pi_{\text{PMatch.Query}}$, $\Pi_{\text{TPSS.XOR}}$, and $\Pi_{\text{aTPSS.AND}}$ algorithms is negligible, the Π_{PInter} algorithm is semantically secure. The security of the Π_{RInter} algorithm can be proved in the same way as that of Π_{PInter} .

C. Security of Our SLIMQ Scheme

Our SLIMQ scheme leaks the ring $\mathbb{Z}_2$, the parameter w , the dimension d , the size of the dataset $|\mathcal{X}|$, the structure of the R-tree T , the prefix size of range query $\{|\mathcal{P}_{[q_{i,l}, q_{i,r}]}\}|_{i=1}^d$, and the access pattern of $\llbracket T \rrbracket$. Based on these leakages, we can simulate the ideal world of our SLIMQ scheme. Since our SLIMQ scheme is implemented using the Π_{PInter} and Π_{RInter} algorithms, the simulation of our SLIMQ scheme can be conducted by calling the simulations of the Π_{PInter} and Π_{RInter} algorithms. The Π_{PInter} and Π_{RInter} algorithms have been proved semantically secure, i.e., the probability that $\{\text{CS}_i\}_{i=1}^3$ can distinguish their views in the real and ideal worlds is negligible; hence, our SLIMQ scheme is also semantically secure.

VI. PERFORMANCE EVALUATION

In this section, we evaluate the performance of our SLIMQ scheme and compare its performance with that of state-of-the-art privacy-preserving multi-dimensional range query schemes, including TRQED+ [20] and PRQ [22]. Both schemes are built on symmetric matrix encryption and effectively protect query privacy in each dimension.

Experimental setting: We implemented our proposed scheme and the compared schemes in C++11, and conducted experiments on a machine equipped with an AMD Ryzen 9 7945HX @2.50 GHz processor, 32 GB RAM, and the Windows 11 operating system. The evaluation was performed on an integrated clinical dataset related to diabetes.¹ Six attributes were selected as the test dataset, including year, age, heart disease, BMI, HbA1c level, and blood glucose level. Some attributes were scaled and shifted to ensure that all data values were positive integers. For the R-tree, the maximum number of child nodes per internal node was set to 8; the security parameter κ was set

¹ The dataset is publicly available at <https://www.kaggle.com/datasets/priyamchoksi/100000-diabetes-clinical-dataset>.

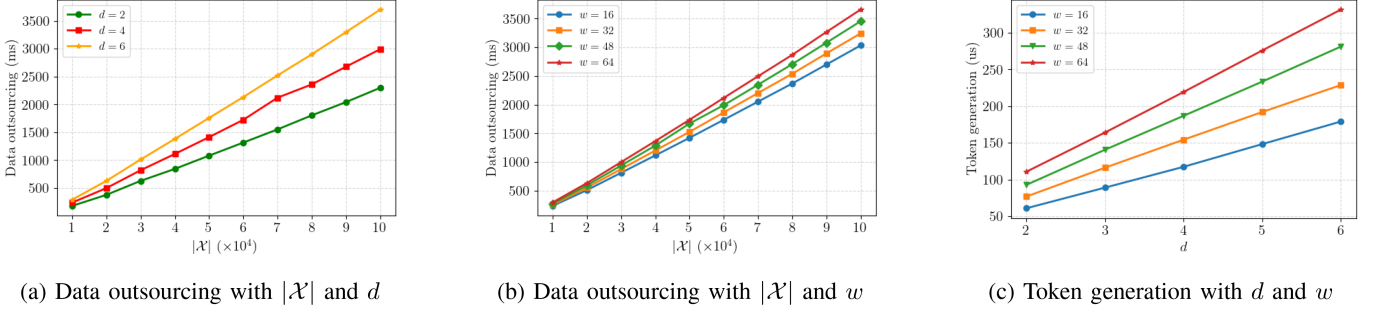


Fig. 2. Runtime performance of data outsourcing and token generation.

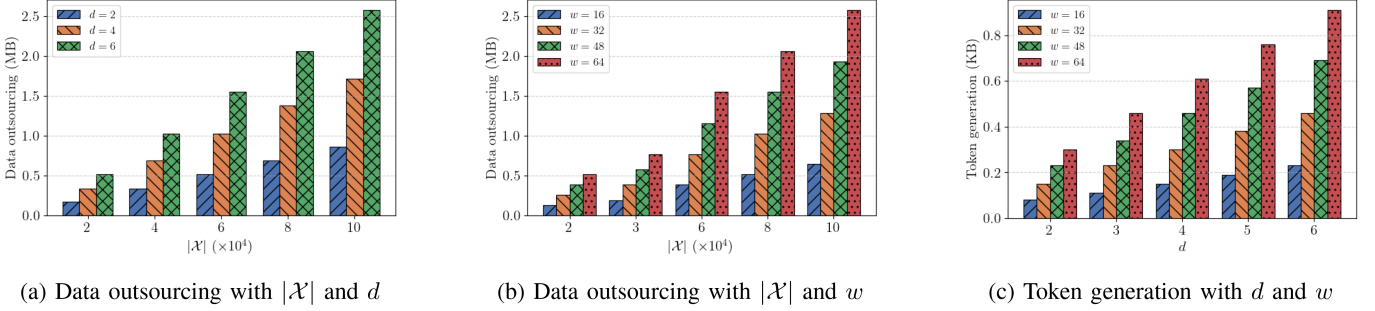


Fig. 3. Communication overhead of data outsourcing and token generation.

to 256. Each experiment was repeated 50 times, and the average results were reported.

In this section, we show how the performance of our scheme varies with different parameters. Our scheme consists of three main phases: data outsourcing, query token generation, and query processing. In the following, we evaluate the performance of our scheme during these phases.

A. Performance of Our Scheme

1) *Data Outsourcing Phase*: The data outsourcing phase mainly consists of two sub-procedures: building the R-tree and sharing the R-tree. The computational complexity of constructing the R-tree T from the dataset $|\mathcal{X}|$ is $O(d \cdot |\mathcal{X}| \cdot \log |\mathcal{X}|)$, while the time complexity of sharing T with $\{CS_i\}_{i=1}^3$ to obtain the encrypted $\llbracket T \rrbracket$ is $O(d \cdot |\mathcal{X}| \cdot w)$. As shown in Fig. 2(a), we illustrate the overall time overhead of data outsourcing as a function of the dataset size $|\mathcal{X}|$ and dimensionality d , with $w = 64$. In Fig. 2(b), we show how the data outsourcing time varies with the dataset size $|\mathcal{X}|$ and w , where $d = 6$. It can be observed that, as $|\mathcal{X}|$ increases, the data outsourcing time grows approximately linearly. Moreover, under the same dataset size, the outsourcing time also increases with higher dimensionality d and larger w .

The communication overhead in the data outsourcing phase arises from sharing T with $\{CS_i\}_{i=1}^3$ to obtain the encrypted $\llbracket T \rrbracket$, which incurs a communication complexity of $O(d \cdot |\mathcal{X}| \cdot w)$. In Fig. 3(a), we show how the communication overhead varies with the dataset size $|\mathcal{X}|$ and dimensionality d , where $w = 64$. In Fig. 3(b), we illustrate its dependence on the dataset size $|\mathcal{X}|$ and parameter w , with $d = 6$. From these two figures, we can

observe that the communication overhead in the data outsourcing phase increases linearly with respect to $|\mathcal{X}|$, dimensionality d , and w .

2) *Token Generation Phase*: In the token generation phase, the computational cost mainly comes from converting each query range $[q_{i,l}, q_{i,r}]$ into a minimal prefix set $\mathcal{P}_{[q_{i,l}, q_{i,r}]}$ and sharing $\mathcal{P}_{[q_{i,l}, q_{i,r}]}$ with cloud servers (where $1 \leq i \leq d$). Since for any query range $[q_{i,l}, q_{i,r}]$, the number of prefixes in its minimal prefix set is at most $2w - 2$, converting $\{[q_{i,l}, q_{i,r}]\}_{i=1}^d$ into $\{\mathcal{P}_{[q_{i,l}, q_{i,r}]}\}_{i=1}^d$ incurs a worst-case computational complexity of $O(d \cdot w^2)$. Similarly, secret-sharing these prefix sets among the servers also incurs a worst-case computational complexity of $O(d \cdot w^2)$. Since the computational cost of these two phases is very low, we repeated each test 10,000 times and took the average value to eliminate the influence of other noise factors. In Fig. 2(c), we illustrate how the time overhead of query token generation varies with the dimension d and the parameter w . It can be clearly observed that the token generation time increases linearly with d , and also grows as w increases.

The communication overhead in the query token generation phase primarily stems from sharing $\mathcal{P}_{[q_{i,l}, q_{i,r}]}$ with $\{CS_i\}_{i=1}^3$, which incurs a worst-case communication complexity of $O(d \cdot w^2)$ bits. In Fig. 3(c), we illustrate how the communication overhead in the token generation phase varies with the dimension d and the parameter w . It can be observed that the communication overhead of the token generation phase increases linearly with d , and also grows as w increases.

3) *Query Processing Phase*: In the query processing phase, $\{CS_i\}_{i=1}^3$ retrieve the query results by recursively traversing the R-tree $\llbracket T \rrbracket$. The computational cost mainly arises from executing the Π_{PInter} and Π_{RInter} algorithms on the internal

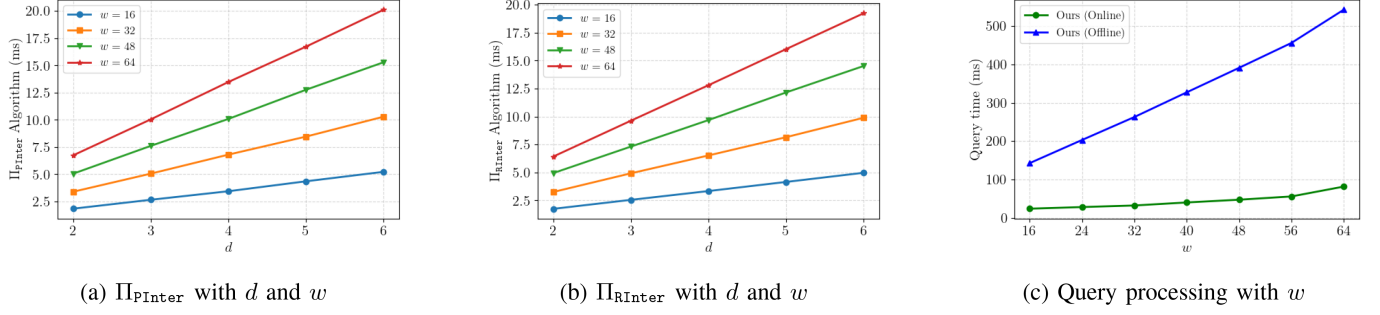


Fig. 4. Runtime performance of query processing.

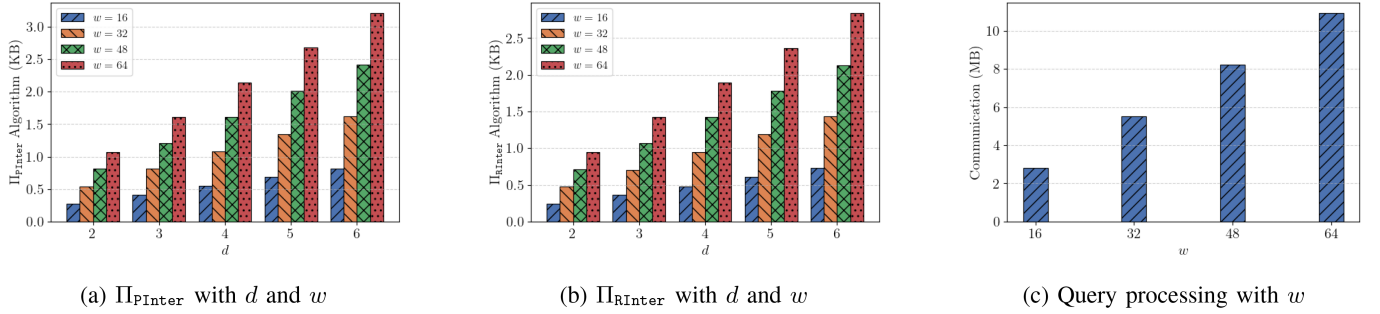


Fig. 5. Communication overhead of query processing.

and leaf nodes. Since the number of prefixes in a prefix set is at most $2w - 2$, the worst-case time complexity of both algorithms is $O(d \cdot w^2)$. Therefore, the overall time complexity of recursively traversing the R-tree $[T]$ is $O(d \cdot w^2 \cdot \log |\mathcal{X}|)$. To facilitate the evaluation of the computational overhead of these two algorithms, in the test of the Π_{PInter} algorithm, the point in each dimension was fixed at 100, and the query range was fixed at $[1, 2020]$; for the Π_{RInter} algorithm, the query range in each dimension was fixed at $[1, 1515]$ and $[505, 2020]$. In Fig. 4(a), we show how the computational cost of the Π_{PInter} algorithm varies with the dimension d and the parameter w ; in Fig. 4(b), we illustrate how the computational cost of the Π_{RInter} algorithm varies with d and w . It can be observed that the computational costs of both algorithms increase linearly with d and grow as w increases. Furthermore, in Fig. 4(c), we present the overall computational overhead of the query processing phase as a function of w , where $|\mathcal{X}| = 100,000$ and $d = 6$. The figure shows that both the offline and online costs grow linearly with w , while the online phase remains significantly more efficient. The computational overhead does not exhibit quadratic growth in practice because the parameter w is typically much smaller than $|\mathcal{X}|$. Consequently, the influence of w on the communication overhead remains relatively limited, which is consistent with the worst-case complexity analysis presented earlier.

The communication overhead in the query processing phase mainly arises from the data exchange between CS_2 and CS_3 in the $\Pi_{aTPSS.AND}([x], [y])$ protocol. Specifically, each AND operation requires CS_2 and CS_3 to exchange 2 bits. Since each node in the R-tree requires $O(d \cdot w^2)$ AND operations for executing Π_{PInter} or Π_{RInter} , and the query processing typically visits $O(\log |\mathcal{X}|)$ nodes during R-tree traversal, the

total communication complexity is $O(d \cdot w^2 \cdot \log |\mathcal{X}|)$ bits. In Fig. 5(a), we show how the communication overhead of the Π_{PInter} algorithm varies with the dimension d and the parameter w ; in Fig. 5(b), we illustrate how the communication overhead of the Π_{RInter} algorithm varies with d and w . It can be observed that the communication overhead of both algorithms increases linearly with d and grows as w increases. Moreover, in Fig. 5(c), we present the total communication overhead of the query processing phase as a function of w , where $|\mathcal{X}| = 100,000$ and $d = 6$. The figure shows that the communication overhead in this phase exhibits an approximately linear growth trend with respect to w .

The computational and communication overhead of the entire query processing phase as a function of the dataset size $|\mathcal{X}|$, dimensionality d , and query result set size will be further discussed in Section VI-B. Therefore, this section focuses exclusively on evaluating the impact of our scheme-specific parameter w on computational and communication overhead.

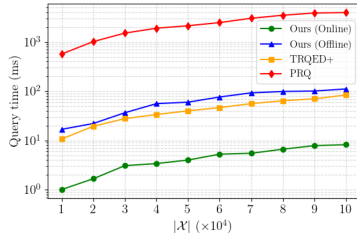
B. Performance Comparison

We compare the SLIMQ scheme with several existing approaches. However, schemes such as [7], [8], [9], [10], [11], [12], [13] either provide relatively weak security, or have query performance that scales linearly with the dataset size $|\mathcal{X}|$, or produce false positives in query results. Therefore, these schemes are not included in our comparison.

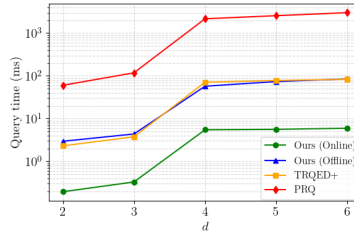
In Table II, we compare our scheme with others across five dimensions: search complexity, basic operation, offline/online support, query privacy, and single-dimensional query privacy. As shown in the table, the Maple scheme [14] and the LSED+

TABLE II
COMPARISON AMONG SLIMQ, PRQ, AND TRQED+

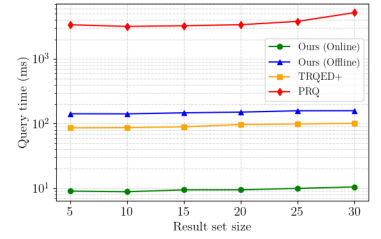
Scheme	Search Complexity	Basic Operation	Offline/Online Support	Query Privacy	Single-Dimensional Query Privacy
Maple [14]	$O(d \cdot \log \mathcal{X})$	Bilinear pairing	×	×	✓
LSED+ [15]	$O(d \cdot (\log N^2)^2 \cdot \log \mathcal{X})$	Bilinear pairing	×	✓	×
Wang et al. scheme [16]	$O(d \cdot \log \mathcal{X})$	Multiplication over $\mathbb{R}$	×	✓	×
Mei et al. scheme [17]	$O(d \cdot \log \mathcal{X})$	Multiplication over $\mathbb{R}$	×	✓	×
TRQED [19]	$O(d \cdot \log \mathcal{X})$	Multiplication over $\mathbb{R}$	×	✓	✓
PRQ [22]	$O(d \cdot N^2 \cdot \log \mathcal{X})$	Multiplication over $\mathbb{R}$	×	✓	✓
TRQED+ [20]	$O(d \cdot \log \mathcal{X})$	Multiplication over $\mathbb{R}$	×	✓	✓
Our SLIMQ	$O(d \cdot w^2 \cdot \log \mathcal{X})$	Multiplication over $\mathbb{Z}_2$	✓	✓	✓



(a) Query processing with $|\mathcal{X}|$



(b) Query processing with d



(c) Query processing with $|\mathcal{A}|$

Fig. 6. Runtime performance comparison of query processing.

scheme [15] exhibit low query efficiency because they are built upon public-key cryptography and rely on bilinear pairings as their core operation. The schemes by Wang et al. [16], Mei et al. [17], and TRQED [19] achieve sublinear query complexity with respect to the dataset size; however, none of them provides full protection for single-dimensional query privacy. Therefore, we select only the two most representative schemes—PRQ [22] and TRQED+ [20]—for a detailed comparison with our scheme.

For the complexity expressions of PRQ and LSED+, the parameter N denotes the square root of the upper bound of values in $\mathcal{X}$. For the PRQ implementation in our experiments, we set $N = 45$ to accommodate the maximum value of 2022 in the dataset. As for TRQED+, its tagged-label mechanism is designed to protect access patterns during R-tree traversal, while SLIMQ does not claim to hide access patterns. Therefore, to compare the core range-query computation under a consistent access-pattern leakage profile, we implement TRQED+ without tagged labels. This tag-disabled version should be regarded as a lower-overhead baseline rather than the full-security TRQED+ scheme. The full TRQED+ scheme provides stronger access-pattern privacy than SLIMQ, but it also introduces additional overhead for tag generation, tag comparison, and oblivious traversal; hence, enabling tagged labels would further increase the overhead of TRQED+. As shown in the table, all three schemes provide single-dimensional query privacy and protect query privacy. However, SLIMQ achieves a query processing complexity of $O(d \cdot w^2 \cdot \log |\mathcal{X}|)$, which is more efficient than PRQ's $O(d \cdot N^2 \cdot \log |\mathcal{X}|)$ under typical settings where $w \ll N$. Although TRQED+ theoretically offers a lower complexity of

$O(d \cdot \log |\mathcal{X}|)$, in practice, when the dataset size is sufficiently large, the parameter w becomes negligible. Moreover, thanks to its offline/online decoupling design, SLIMQ achieves significantly higher online query efficiency in real-world deployments, offering a clear advantage over other schemes. Furthermore, the basic operation in our scheme is multiplication over $\mathbb{Z}_2$, which is considerably faster than multiplication over $\mathbb{R}$ used in the other approaches. Since the performance of TRQED+, PRQ, and our scheme all depends on the parameters d and $|\mathcal{X}|$, we compare the query time and communication overhead of our scheme against TRQED+ and PRQ under varying values of d and $|\mathcal{X}|$.

In Fig. 6(a), we show how the query time of the three schemes varies with the dataset size $|\mathcal{X}|$, where $d = 6$ and $w = 11$. It can be seen that the time overhead of all three schemes exhibits a sublinear growth trend as $|\mathcal{X}|$ increases. Fig. 6(b) shows how the query time of the three schemes varies with the dimension d , where $|\mathcal{X}| = 100,000$ and $w = 11$. It can be observed that the time overhead of all three schemes increases linearly with d . Since all three schemes rely on R-trees for retrieval, they are not well-suited for high-dimensional data. They achieve very high efficiency when $d < 4$, but performance degrades as d exceeds 4, leading to noticeable fluctuations in the results shown in the figure. Fig. 6(c) further evaluates how the query time of the three schemes varies with the size of the query result set $|\mathcal{A}|$, where $d = 6$, $|\mathcal{X}| = 100,000$, and $w = 11$. It can be seen that, as the number of query results increases, the overhead of all three schemes exhibits a mild linear growth trend. From the above three figures, it can be observed that the relative trends of the query time for the three schemes are consistent. The computational overhead of our scheme in the online phase shows

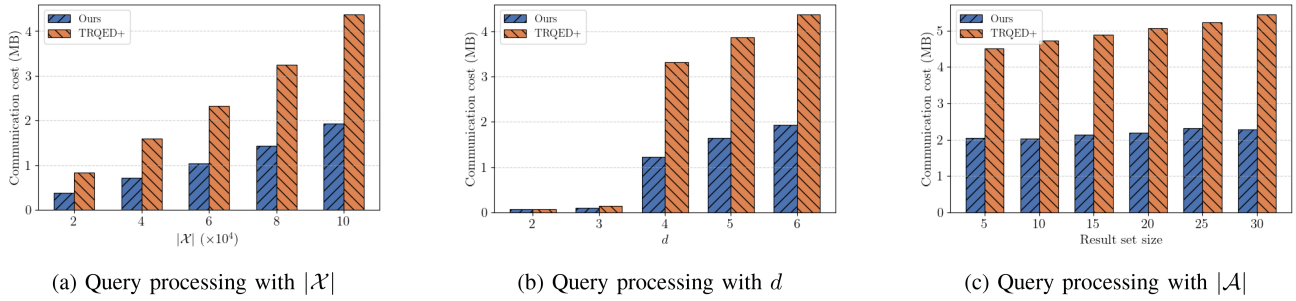


Fig. 7. Communication overhead comparison of query processing.

a significant performance advantage, achieving more than an order-of-magnitude improvement compared with the TRQED+ scheme. Meanwhile, the computational efficiency in the offline phase is also satisfactory, with a computational overhead comparable to that of the TRQED+ scheme.

In terms of communication overhead, since the PRQ scheme employs a single-cloud model and incurs no communication cost during query processing, we compare our scheme only with TRQED+. In Fig. 7(a), we show how the communication overhead during query processing varies with the dataset size $|\mathcal{X}|$, where $d = 6$ and $w = 11$. In Fig. 7(b), we illustrate how this overhead varies with the dimension d , with $|\mathcal{X}| = 100,000$ and $w = 11$. Finally, in Fig. 7(c), we present the communication overhead of query processing as a function of the query result set size, where $d = 6$, $|\mathcal{X}| = 100,000$, and $w = 11$. As shown in these three figures, the communication overhead of both our scheme and TRQED+ grows linearly with respect to $|\mathcal{X}|$, d , and the size of the query result set. Moreover, the communication overhead of our scheme is approximately half that of TRQED+, demonstrating a clear advantage.

VII. RELATED WORK

In recent years, how to support privacy-preserving multi-dimensional range queries over encrypted data has attracted significant attention. Researchers have proposed various solutions aiming to balance query efficiency and data privacy.

Methods based on Order-Preserving Encryption (OPE) [8], [9] can naturally support range queries because they preserve the order relations among plaintexts during encryption, allowing the server to directly perform comparison operations over ciphertexts. However, this order-preserving property may also introduce potential information leakage risks. Existing studies have shown that OPE schemes cannot guarantee strong security under ordered chosen plaintext attacks [29]. Even if an adversary cannot recover the exact plaintext values, it may still infer the underlying structure of the original data by analyzing the relative order, distribution characteristics, or statistical information of ciphertexts, which limits the level of privacy protection that such methods can provide.

To mitigate the leakage of order information, several studies have proposed range query methods based on bucketization [10], [11]. These approaches partition the plaintext domain into several discrete intervals and map data values to

the corresponding buckets, thereby concealing the exact numerical values to a certain extent. Specifically, scheme [10] proposed a secure range query mechanism based on bucketized encoding; however, this scheme only supports one-dimensional data. Subsequently, scheme [11] extended the bucketization idea to multi-dimensional scenarios and designed an optimization strategy to balance the trade-off between query processing overhead and information leakage. Nevertheless, such methods still suffer from several limitations. First, due to the granularity introduced by bucket partitioning, the query results may contain records that do not satisfy the query condition, leading to the problem of false positives. Second, additional computations such as result filtering and verification are typically performed by the querying user, which increases the computational burden on the client side.

In addition to symmetric encryption methods, some studies employ public-key cryptography to construct privacy-preserving multi-dimensional range query mechanisms [12], [13], [14]. Among them, scheme [12] proposed a query scheme based on Hidden Vector Encryption (HVE), which supports multi-dimensional range condition matching, where query predicates are expressed through vector encoding and inner-product predicates. Scheme [13] decomposed multi-dimensional range queries into multiple one-dimensional interval queries and combined them with an interval-tree index structure to achieve secure query processing under a bilinear pairing-based public-key encryption framework. Furthermore, scheme [14] integrated HVE with the R-tree index structure to support multi-dimensional range queries over spatial data. Although these schemes provide strong formal security guarantees, their overall efficiency is often insufficient for large-scale practical applications due to the high computational overhead introduced by bilinear pairing operations.

To further improve query efficiency, researchers have proposed a series of more lightweight solutions [15], [16], [17]. For example, scheme [15] adopts symmetric-key range predicate encryption and utilizes a segment-tree structure to encode the numerical domain, thereby enabling logarithmic-complexity one-dimensional range queries in cloud environments. Scheme [16] combines asymmetric scalar-product-preserving encryption with the R-tree index structure to support multi-dimensional range query operations. Scheme [17] constructs an encrypted interval tree index structure and integrates ASPE with a noise mechanism [18] to enhance

data privacy protection, thereby enabling flexible sub-range queries in multi-user scenarios. Although these schemes significantly improve system performance by reducing expensive public-key operations and introducing efficient index structures, most of them suffer from the problem of single-dimensional query privacy leakage.

To further strengthen privacy protection, some studies have begun to focus on protecting single-dimensional query privacy. For example, the TRQED scheme proposed in [19] achieves multi-dimensional range queries by combining the R-tree index structure and employs a matrix encryption technique with fixed-dimension permutation to enable secure multi-dimensional range queries with partial single-dimensional query privacy protection. The TRQED+ scheme [20] further improves TRQED by introducing a secure inner-product computation protocol [21] and an oblivious R-tree traversal algorithm, which not only protects data privacy and query privacy, but also hides the access pattern during query processing. The PRQ scheme proposed in [22] adopts a data-encoding technique that transforms data comparison operations into equality tests. Combined with matrix encryption, it designs point/range intersection test protocols that can fully protect single-dimensional query privacy, thereby supporting secure multi-dimensional range queries based on the R-tree. Nevertheless, this method heavily depends on the size of the numerical domain and requires a predefined domain parameter N to construct the encoding matrix, where N^2 represents the upper bound of the numerical domain. When the data domain is large, the matrix size grows rapidly, resulting in significant storage and computational overhead. In addition, some recent schemes [30], [31], [32] have further studied verifiable multi-dimensional range queries, respectively focusing on multi-user verifiability, dynamic spatial data, and encrypted geospatial data. Compared with these verifiable schemes, SLIMQ does not aim to provide result verification against malicious servers, but focuses on achieving full single-dimensional query privacy and lightweight online query processing in the semi-honest outsourced query setting.

Different from existing schemes, our SLIMQ scheme can achieve full single-dimensional query privacy protection while maintaining high query efficiency.

VIII. CONCLUSION

In this paper, we propose SLIMQ, a secure lightweight online/offline multi-dimensional range query scheme. In our scheme, we first organize data using an R-tree index structure, and then employ prefix encoding techniques to transform the R-tree-based multi-dimensional range query problem into a prefix matching problem, thereby enabling efficient multi-dimensional range query processing. Subsequently, based on existing three-party secret sharing schemes, we further propose an adaptive three-party secret sharing scheme, upon which we construct the Π_{PInter} algorithm and the Π_{RInter} algorithm. These algorithms adopt an offline-online separation paradigm, allowing computation-intensive tasks to be performed in the offline phase, while the online phase only involves lightweight operations. Security analysis shows that SLIMQ can provide strong

security guarantees under the semi-honest adversarial model and can fully protect single-dimensional query privacy. Extensive experimental evaluations demonstrate that SLIMQ significantly outperforms existing privacy-preserving range query schemes (such as TRQED+ and PRQ) in terms of online query efficiency, while maintaining competitive efficiency in the offline phase. In future work, we plan to extend SLIMQ to support dynamic data updates, malicious adversarial models, and large-scale federated query scenarios.

REFERENCES

- [1] S. Dash, S. K. Shakyawar, M. Sharma, and S. Kaushik, "Big data in healthcare: Management, analysis and future prospects," *J. Big Data*, vol. 6, no. 1, pp. 1–25, 2019.
- [2] K. Shahzad et al., "Big data analytics in healthcare: Current practices, innovations, and future prospects," *J. Big Data*, vol. 12, no. 1, pp. 1–33, 2025.
- [3] M. Babar, M. A. Jan, X. He, H. Deng, W. Chen, and M. J. Zemerly, "An optimized IoT-enabled Big Data analytics architecture for edge-cloud computing," *IEEE Internet Things J.*, vol. 10, no. 5, pp. 3995–4005, Mar. 2023.
- [4] R. Lu, "A new communication-efficient privacy-preserving range query scheme in fog-enhanced IoT," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2497–2505, Apr. 2019.
- [5] P. Mell and T. Grance, "The NIST definition of cloud computing," *Nat. Inst. Standards Technol.*, Special Publication 800-145, National Institute of Standards and Technology, Gaithersburg, MD, USA, Sep., 2011.
- [6] S. Kamara, C. Papamanthou, and T. Roeder, "Dynamic searchable symmetric encryption," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2012, pp. 965–976.
- [7] R. Agrawal, J. Kiernan, R. Srikant, and Y. Xu, "Order preserving encryption for numeric data," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2004, pp. 563–574.
- [8] A. Boldyreva, N. Chenette, Y. Lee, and A. O'Neill, "Order-preserving symmetric encryption," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2009, pp. 224–241.
- [9] A. Boldyreva, N. Chenette, and A. O'Neill, "Order-preserving encryption revisited: Improved security analysis and alternative solutions," in *Proc. Annu. Cryptol. Conf.*, 2011, pp. 578–595.
- [10] Y. Lee, "Secure ordered bucketization," *IEEE Trans. Dependable Secur. Comput.*, vol. 11, no. 3, pp. 292–303, May/Jun. 2014.
- [11] B. Hore, S. Mehrotra, M. Canim, and M. Kantarcioglu, "Secure multidimensional range queries over outsourced data," *VLDB J.*, vol. 21, no. 3, pp. 333–358, 2012.
- [12] D. Boneh and B. Waters, "Conjunctive, subset, and range queries on encrypted data," in *Proc. Theory Cryptogr. Conf.*, S. P. Vadhan, Ed., 2007, pp. 535–554.
- [13] E. Shi, J. Bethencourt, T. H. Chan, D. X. Song, and A. Perrig, "Multidimensional range query over encrypted data," in *Proc. IEEE Symp. Secur. Privacy*, 2007, pp. 350–364.
- [14] B. Wang, Y. Hou, M. Li, Y. Zhu, and K. Ren, "Maple: Scalable multi-dimensional range search over encrypted cloud data with tree-based index," in *Proc. 9th ACM Symp. Inf., Comput. Commun. Secur.*, 2014, pp. 111–122.
- [15] Y. Lu, "Privacy-preserving logarithmic-time search on encrypted data in cloud," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2012, pp. 1–17.
- [16] P. Wang and C. V. Ravishanker, "Secure and efficient range queries on outsourced databases using RP-trees," in *Proc. IEEE 29th Int. Conf. Data Eng. Workshops*, 2013, pp. 314–325.
- [17] Z. Mei et al., "Executing multi-dimensional range query efficiently and flexibly over outsourced ciphertexts in the cloud," *Inf. Sci.*, vol. 432, pp. 79–96, 2018.
- [18] S. Su, Y. Teng, X. Cheng, K. Xiao, G. Li, and J. Chen, "Privacy-preserving TOP-k spatial keyword queries in untrusted cloud environments," *IEEE Trans. Servi. Comput.*, vol. 11, no. 5, pp. 796–809, Sep./Oct. 2018.
- [19] W. Yang, Y. Xu, Y. Nie, Y. Shen, and L. Huang, "TRQED: Secure and fast tree-based private range queries over encrypted cloud," in *Proc. Int. Conf. Database Syst. Adv. Appl.*, 2018, pp. 130–146.
- [20] W. Yang, Y. Geng, L. Li, X. Xie, and L. Huang, "Achieving secure and dynamic range queries over encrypted cloud data," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 1, pp. 107–121, Jan. 2022.

- [21] M. J. Atallah and W. Du, "Secure multi-party computational geometry," in *Proc. Workshop Algorithms Data Structures*, 2001, pp. 165–179.
- [22] Y. Zheng, R. Lu, Y. Guan, J. Shao, and H. Zhu, "Towards practical and privacy-preserving multi-dimensional range query over cloud," *IEEE Trans. Dependable Secur. Comput.*, vol. 19, no. 5, pp. 3478–3493, Sep./Oct. 2022.
- [23] R. Li, A. X. Liu, H. Xu, Y. Liu, and H. Yuan, "Adaptive secure nearest neighbor query processing over encrypted data," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 91–106, Jan./Feb. 2022.
- [24] H. Chaudhari, A. Choudhury, A. Patra, and A. Suresh, "ASTRA: High throughput 3PC over rings with application to secure prediction," in *Proc. ACM SIGSAC Conf. Cloud Comput. Secur. Workshop*, 2019, pp. 81–92.
- [25] Y. Zheng, R. Lu, S. Zhang, H. Zhu, and F. Wang, "Efficiency-improved privacy-preserving weighted similarity query over outsourced ehealthcare data," in *Proc. IEEE Glob. Commun. Conf.*, 2022, pp. 4866–4871.
- [26] Y. Zheng et al., "Towards efficient and secure hypercube tree building for vertically distributed data in cloud," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 9025–9038, 2025.
- [27] N. Beckmann, H.-P. Kriegel, R. Schneider, and B. Seeger, "The R*-tree: An efficient and robust access method for points and rectangles," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 1990, pp. 322–331.
- [28] Y. Zheng et al., "Epset: Efficient and privacy-preserving set similarity range query over encrypted data," *IEEE Trans. Servi. Comput.*, vol. 17, no. 2, pp. 524–536, Mar./Apr. 2024.
- [29] L. Xiao and I. Yen, "A note for the ideal order-preserving encryption object and generalized order-preserving encryption," *IACR Cryptol. ePrint Arch.*, vol. 2012, 2012, p. 350.
- [30] H. Wang, K. Fan, K. Zhang, F. Li, H. Li, and Y. Yang, "MU-MRQ: Enabling multi-user verifiable and secure multi-dimensional range query over encrypted data," *IEEE Trans. Dependable Secur. Comput.*, vol. 22, no. 6, pp. 7773–7789, Nov./Dec. 2025.
- [31] H. Wang, K. Fan, Y. Quan, F. Li, and H. Li, "VDPSRQ: Achieving verifiable and dynamic private spatial range queries over outsourced database," in *Proc. 23rd IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun.*, Sanya, China, 2024, pp. 532–541.
- [32] Z. Mei et al., "Efficient and verifiable range query scheme for encrypted geographical information in untrusted cloud environments," *ISPRS Int. J. Geo Inf.*, vol. 13, no. 8, 2024, Art. no. 281.